

Image File Read and Write

Image File Read and Write

Preface

Example Results

Theory

File Read/Write vs Memory Encoding/Decoding

Pre-Run Preparation

Code Overview

Import Modules

Read and Save Image from SD Card

Create Test Image and Encode as PNG

Decode from Memory Buffer

Set JPEG Quality

Run Existing Tests

Parameter Adjustment Notes

Complete REPL Exercise

Preface

This tutorial applies to firmware version: CanMV_K230_YAHBOOM_micropython_v1.8-13. You can download it from the Kendryte official website at https://download.kendryte.com/developer/releases/canmv_k230_micropython/daily_build/



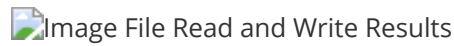
Example Results

This section is not a real-time camera loop but rather completes SD card image reading, saving, and memory encoding/decoding in K230 REPL or scripts. Reference code:

- [demo_basic_ops.py](#): imread(), imwrite().
- [test_imgcodecs_memory.py](#): imencode(), imdecode().

After running, you should see output files in /sdcard and encoding buffer length or test pass information in the terminal.

Screenshot the IDE terminal and SD card output file list, and save as 1.png to the current directory.

Image File Read and Write Results

Theory

File Read/Write vs Memory Encoding/Decoding

Function	Input	Output	Typical Use
simread()§	File path	Image array	Load images from SD card
simwrite()§	File path, image array	Success flag	Save processing results to file
simencode()§	Extension, image array	Success flag, byte buffer	Network transmission, memory cache
simdecode()§	Encoded buffer	Image array	Restore image from received data

imread() and imwrite() access the file system. imencode() and imdecode() only convert between raw pixels and compressed formats like PNG/JPEG in memory.

Pre-Run Preparation

1. Confirm SD card is mounted as /sdcard.
2. If reading existing images, name the test image test.jpg and place it in /sdcard.
3. Confirm remaining storage space is sufficient for output files.
4. JPEG encoding/decoding generates extra buffers. Run gc.collect() before execution.

Code Overview

Import Modules

```
import cv2
from ulab import numpy as np
import gc
```

This section does not require initializing Sensor or Display.

Read and Save Image from SD Card

```
img = cv2.imread("/sdcard/test.jpg")

if img is None:
    print("/sdcard/test.jpg not found")
else:
    print("Image size:", img.shape)
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    success = cv2.imwrite("/sdcard/gray.png", gray)
    print("Write result:", success)
```

When reading fails, `imread()` returns `None`. You must check this before calling other processing functions. `imwrite()` return value confirms whether encoding and file writing succeeded.

Create Test Image and Encode as PNG

```
img = np.zeros((60, 80, 3), dtype=np.uint8)
cv2.circle(img, (40, 30), 25, (0, 255, 0), -1)
cv2.rectangle(img, (10, 10), (30, 20), (255, 0, 0), -1)

success, buffer = cv2.imencode(".png", img)
if not success:
    raise RuntimeError("PNG encoding failed")

print("PNG buffer length:", len(buffer))
```

The extension determines the encoding format. `buffer` is compressed memory data, not the original `HxWxC` pixel array.

Decode from Memory Buffer

```
decoded = cv2.imdecode(buffer, cv2.IMREAD_COLOR)
if decoded is None:
    raise RuntimeError("Image decoding failed")

print("Decoded size:", decoded.shape)
```

`IMREAD_COLOR` requests color output; `IMREAD_GRAYSCALE` requests grayscale output. Decode mode affects the array channel count.

Set JPEG Quality

```
success, jpeg_buffer = cv2.imencode(
    ".jpg",
    img,
    [1, 50]
)
```

The current test code uses parameter number 1 to indicate JPEG quality with a value of 50. Higher quality usually means larger file size and less compression loss. Parameter behavior should be verified based on current firmware testing.

Run Existing Tests

```
import sys
sys.path.append("/sdcard")
import test_imgcodecs_memory
test_imgcodecs_memory.run_all()
```

You need to place the test script in /sdcard first, or in MicroPython's importable path. Existing tests cover PNG encoding/decoding, JPEG quality parameters, and grayscale round-trip.

Parameter Adjustment Notes

- **Paths:** K230 uses forward slash paths. It is recommended to explicitly write /sdcard/filename.
- **Read failure:** First check filename case sensitivity, extension, mount status, and format support. Do not directly access `None.shape`.
- **PNG vs JPEG:** PNG is lossless but may be slower and larger. JPEG is lossy, suitable for color photos and transmission.
- **Grayscale image shape:** Different interfaces may use HxW or HxWx1. Check shape before passing to the next function.
- **Memory peak:** During encoding, the original image and compressed buffer may be held simultaneously. During decoding, the compressed buffer and output image may be held simultaneously. For large images, execute items one at a time and release unused references promptly.
- **Write lifespan:** Do not write to SD card indiscriminately in high frame rate loops to avoid affecting real-time performance and storage medium lifespan.

Complete REPL Exercise

```
import cv2
import gc

img = cv2.imread("/sdcard/test.jpg")
if img is None:
    print("Read failed")
else:
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    print("Save grayscale:", cv2.imwrite("/sdcard/gray.png", gray))

    ok, buf = cv2.imencode(".png", gray)
    print("Memory encode:", ok, "length:", len(buf) if ok else 0)

    if ok:
        restored = cv2.imdecode(buf, cv2.IMREAD_GRAYSCALE)
        print("Memory decode:", restored is not None)
```

```
gc.collect()
```